
Abstractification

Alperen Keles
alpkeles99@gmail.com

Abstract

Large language models can rapidly generate and modify programs for a given specification, leading to a verification bottleneck in the software development cycle. We present *abstractification*, a programming model using program sketching that separates the specification of a program from its implementation, requiring human review only at specification boundaries. Abstractified programs contain *holes* that a model is free to change, mechanically derived or human-authored *verifiers* that evaluate any proposals to the holes, and the surrounding program that is part of the specification. We propose an autonomous optimization agent that can use manually written benchmarks, performance fuzzing and recorded inputs for optimization by autonomously searching through optimized candidates for holes in a program in a CEGIS-style program synthesis loop. We prototype abstractification in Rust using macro annotations for reference implementations, contracts, data models, and optimization objectives. Our preliminary study of allocation heuristics in Rust’s `str::replacen` illustrates that the best implementation choice can significantly depend on the production workload, and that the hole-oriented approach can allow for the model to autonomously search for optimizations without per-proposal human review within the explicitly defined search space.

1 Introduction

Program synthesis has classically been formulated as a search problem: given a specification, find a program satisfying it in the space of all programs of a language[Gulwani et al., 2017]. Specifications may take many forms: logical formulas[Kuncak et al., 2010], the input-output examples of programming-by-example[Gulwani, 2011], reference implementations... The space of possible programs is further narrowed by structural constraints such as program sketches[Solar-Lezama et al., 2006] or language grammars[Alur et al., 2013]. The search itself has evolved from enumerative and constraint-based methods[Gulwani et al., 2017, Kuncak et al., 2010] through counterexample-guided inductive synthesis[Jha et al., 2010] to, most recently, large language models, which turn natural language and surrounding code into candidate programs at previously unattainable scale[Li et al., 2022]. A successful search, by itself, does not guarantee optimality of the synthesized program. For instance, the simplest result of a programming-by-example synthesis is a program that encodes the examples into the program, or in other words a program that overfits to the given examples. Thus, the synthesis typically includes a ranking to pick the "best" program in the space of accepted programs[Gulwani, 2011, Mayer et al., 2015].

The rise of large language models and their success in program synthesis from natural language specifications have shifted the program synthesis from a research effort into a mainstream programming tool. Practitioners have been dealing with the hard-earned lessons of the synthesis literature from under-specification to mis-specification to failure of generalization[Yang et al., 2025, Amodei et al., 2016, Devlin et al., 2017, Mayer et al., 2015]. The current response to these problems is two-fold. The first is *alignment*[Amodei et al., 2016], large language models are much more powerful than naive code synthesizers. LLMs can ingest arbitrary streams of information, invoke tools that can search or produce local and online data, can ask questions to clarify the user intent, and can do all these in a continuous, incremental "agentic" loop. The reliability in which the model can achieve these tasks

is one aspect the programmer must work with, but whether the model pursues the same goal as the user depends on how *aligned* it is. The second is a human-in-the-loop (HITL) verification process, the result of which is the "verification bottleneck"[Sonar, 2026]. The HITL process starts with an initial intent expressed in a combination of natural language and code, formalized and implemented by the agent to produce a verifiable artifact to be accepted, rejected, and refined by the programmer. The verifiable artifact itself can sometimes be an executable specification that the model can use for further development, such as the initial conversation producing a set of test cases to be verified by the user, which is then used in an autonomous loop as the model hill-climbs the number of passing test cases.

This type of hill-climbing can be very powerful, attested by the success of AlphaEvolve[Novikov et al., 2025], popular hill-climbing strategies based on auto-research[Karpathy, 2026], and the surrounding literature on autonomous systems optimization[Romera-Paredes et al., 2023]. However, to our knowledge, although there are many existing systems for *proposing* optimizations, implementation of these proposed optimizations is rarely autonomous; the human in the loop prevails. *This is very surprising!* Programmers have been using optimizing compilers[Lattner and Adve, 2004], just-in-time (JIT) compilers[Bala et al., 2000], even super-optimizers[Massalin, 1987] in narrow scenarios for a very long time without manually reviewing the optimized assembly. What makes the current generation of optimizers different? Why do they require additional manual review?

The first answer to these questions is that the existing compiler optimizations are well tested, sometimes even proven[Leroy, 2009, Lopes et al., 2021] algorithmic passes. An optimization assumes a formal cost model, in which it looks for patterns of control and data flow with equivalent semantics with a lower cost. The optimization pass is then evaluated with respect to the cost model, the implementation of the pass is continuously tested on every program compilation as well as extensively tested as part of the compiler testing procedures, incrementally resolving any uncaught bugs until it is production-ready and usable without a human in the loop; a scenario very much in contrast to the neural program synthesis. The second answer, which marks the core idea of this paper, is that autonomous optimizations are only possible within completely specified semantics.

A complete specification fully defines what is guaranteed as well as what is not. For instance, a programming language defines a computation at a particular granularity with respect to memory layout, memory management, order of execution... The compiler is then free to optimize within the space of possible machine code that corresponds to the defined semantics. The reason that the current generation of optimizers require manual additional review is that they do not have access to such a space of defined semantics. The neural optimizer targets the programming language, but any change to the program is potentially a behavioral change, the specification and the implementation are coupled within the same program.

In this paper, we propose a method for decoupling the specification of a program from its implementation that we call **abstractification**. An abstractified program denotes not one implementation but the set of implementations that fit its sketch and satisfy its specification. An autonomous optimizer is then free to search that set, evaluate the candidates it finds, and adopt any member: every member satisfies the specification, so no individual choice requires human review. Abstractification is not one technique, but an array of techniques as building blocks for a higher level programming language where large language models act as optimizing compilers.

The rest of this paper is as follows: §2 puts abstractification in context of existing literature and prior works; §3 explains the set of abstractifications we propose along with verification methods we use; in §4 we showcase a small autonomous optimization case study using abstractification; §5 presents similar works in the literature; and in §6 we present our thoughts on the future work and implementation of abstractification.

2 Background

Abstractification is an amalgamation of existing literature on program synthesis, program verification, and neural optimizers.

Counterexample Guided Inductive Synthesis (CEGIS). CEGIS[Jha et al., 2010, Solar-Lezama, 2008] is an incremental program synthesis strategy that combines a candidate program synthesizer with a verifier that produces a counterexample when the program does not match the specification. At

each step, the synthesizer proposes a candidate consistent with all counterexamples gathered so far; the verifier either certifies it against the specification or returns a fresh counterexample, illustrated in Fig. 1. In abstractification, the synthesizer is a large language model, or an agentic harness built around one, that receives counterexamples or other rejection feedback from the verification strategies described in §3.

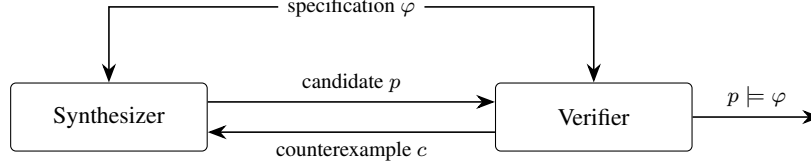


Figure 1: CEGIS loop.

Program sketching. Program sketching[Solar-Lezama et al., 2006] is a synthesis technique that the programmer writes a program with holes alongside a partial specification, and delegates filling the holes to counterexample-guided inductive synthesis[Jha et al., 2010]. Sketching demonstrated that programmers can productively communicate *insight* while omitting *detail*. Fig 2 presents synthesis of an optimized bit manipulation for isolating the rightmost zero bit of a word. In abstractification, we invert the workflow: we begin from a complete concrete program and *carve* holes into it.

```

bit[W] isoSpec(bit[W] x) {                // executable specification: the
    bit[W] ret = 0;                        // obvious loop, correct but slow
    for (int i = 0; i < W; i++)
        if (!x[i]) { ret[i] = 1; return ret; }
    return ret;
}

bit[W] isolate0(bit[W] x) implements isoSpec {
    return ~(x + ??) & (x + ??);           // sketch: the shape, without detail
}                                           // solver fills ?? = 0 and ?? = 1

```

Figure 2: A program sketch isolating the rightmost zero bit of a word, adapted from Solar-Lezama’s dissertation[Solar-Lezama, 2008, pp. 24–25].

Design by contract. Design by contract (DbC)[Meyer, 1992] is a specification technique where preconditions, postconditions, assertions, and invariants are written within modules, functions, and classes. DbC has originated in Eiffel[Meyer, 1992], the idea has been widely adapted: runtime checking in JML[Leavens et al., 2006] and higher-order contracts in Racket[Findler and Felleisen, 2002]; static, SMT-backed verification in Spec# [Barnett et al., 2005] and Dafny[Leino, 2010]; static and dynamic contract libraries in Haskell, Rust, Python... These systems historically treated the contracts as a validation of a fixed implementation, we invert this frame to treat contracts as synthesis boundaries similar to sketches.

Random testing. Property-based testing (PBT) is a random testing technique popularized by QuickCheck[Claessen and Hughes, 2000] where users write executable specifications in the form of universally quantified first order logical formulas checked against randomly generated inputs. Abstractification leverages contracts, properties and reference implementations as specifications to synthesize against; we propose using PBT, fuzzing and formal verification as verifiers in the CEGIS loop. A crucial advantage of random testing over formal verification is that it is not limited to formally modeled domains, which allows us to use the same testing harnesses for performance testing for optimization as well.

3 Methodology

Our proposal consists of three components. The first is a set of language constructs for abstractifying programs. The second is a set of testing and verification strategies for leveraging abstractifications in

a program. §3.1 presents how we can abstractify programs, creating the specification as the target of program synthesis, while §3.2 presents techniques for deriving verifiers to use within the CEGIS loop. The last component is introducing a language model for proposing and leveraging abstraction, the details of which we depict in §3.3.

3.1 Methods of Abstractification

The simplest form of abstractification is a reference implementation as shown in Fig. 2. The example perfectly shows how sketching allows for expressing semantics at multiple granularity in the same language. The author could simply mark the whole body of `isolate0` as a hole, but instead provides the general shape, expecting the synthesis to incrementally discover the correct mask values. We use the Rust macro system for implementing abstractifications, Fig. 3 presents our version of Fig. 2:

```
fn isolate0_spec(x: u32) -> u32 {
    for i in 0..32 {
        if x & (1 << i) == 0 { return 1 << i; }
    }
    0
}

#[abstractified(reference = "isolate0_spec")]
fn isolate0(x: u32) -> u32 {
    !(x.wrapping_add(hole!())) & x.wrapping_add(hole!())
    // CEGIS fills 0 and 1
}
```

Figure 3: Sketching in Rust.

The holes can take arguments such as `hole!(0)` that marks an existing potential value to be replaced. These types of holes are useful for parameter tuning in programs instead of synthesizing correct implementations with respect to reference implementations or tests.

Examples of these parameters can be architectural heuristics, anticipated common user behaviors such as small string/vector optimizations, or sometimes bespoke parameters resulting from parameter search methods. One example we picked is the `replacen` function from the Rust standard library [Zeng and Rust Project Contributors, 2016], presented in Fig. 4.

```
pub fn replacen<P: Pattern>(&self, pat: P, to: &str, count: usize) -> String {
    // Hope to reduce the times of re-allocation
    let mut result = String::with_capacity(32);
    let mut last_end = 0;
    for (start, part) in self.match_indices(pat).take(count) {
        result.push_str(unsafe { self.get_unchecked(last_end..start) });
        result.push_str(to);
        last_end = start + part.len();
    }
    result.push_str(unsafe { self.get_unchecked(last_end..self.len()) });
    result
}
```

Figure 4: `replacen` from Rust Standard Library.

The *hope* seems to have survived the test of time, making this heuristic a useful case study for abstractification. The initial capacity does not change the returned string when execution succeeds, but it remains observable through allocation failure, memory consumption, and execution time. This separation creates a narrowly scoped optimization hole: the verifier checks output equivalence, while the optimizer searches for a distribution-dependent capacity expression. In §4, we evaluate whether agents can optimize this hole autonomously across several input distributions.

The second type of abstractification we propose is data abstraction. Programming languages have had data abstraction for a very long time in the form of inheritance in object-oriented programming [Dahl and Nygaard, 1966], modules in ML [MacQueen, 1984], type classes in Haskell [Wadler and Blott, 1989] or traits in Rust [Jung et al., 2017]. Abstract objects can be specialized at compile time for static

dispatch or dynamically dispatched at runtime; but the caller must define the concrete object at some point in the call tree. We propose allowing programmers to directly write abstract objects in code, the synthesizer picking an existing implementation of the interface at synthesis time, optimized with respect to feedback from the verifiers. We illustrate an abstract counter map in Fig. 5.

```
interface!(CountMap {
  model: BTreeMap<String, usize>,
  observe: |m| { let mut v: Vec<_> = m.into_iter().collect();
                v.sort(); v },
  ops: { bump = |m, k: String| *m.entry(k).or_default() += 1 },
});

// Abstract map object
let mut counts = hole!(CountMap, HashMap::::new());
// Alternatively with no default
let mut counts_ = hole!(CountMap);
```

Figure 5: Definition of an abstract CountMap and its usage with a default HashMap.

3.2 Methods of Verification

An abstractification combines a hole with an executable specification constrained by types, assertions, and data abstractions. When the synthesizer proposes a concrete implementation for an abstractified hole, the verifier must accept or reject the proposal.

We distinguish the artifacts that specify intended behavior from the mechanisms that verify a candidate against them. A reference implementation can serve as an executable oracle: the verifier runs both implementations on generated or recorded inputs and compares their outputs. Preconditions, postconditions, and assertions can be checked dynamically during testing or, when expressed in Verus[Lattuada et al., 2023], become proof obligations. Recorded input-output pairs support replay tests that check whether a candidate preserves previously observed behavior.

These mechanisms provide different levels of assurance. Formal verification can establish that a candidate satisfies a formalized specification, property-based testing searches generated inputs for counterexamples, and replay testing checks equivalence only over the recorded cases. Once a candidate passes the applicable correctness verifiers, performance measurements rank it against other accepted candidates. We propose performance fuzzing [Petsios et al., 2017, Lemieux et al., 2018] and benchmarks over recorded production traffic for this ranking step.

Our current prototype automatically synthesizes proptest[proptest-rs contributors, 2026] harnesses for abstractified functions and uses hand-written benchmarks for performance ranking. Agent-driven Verus verification, performance fuzzing, and production-traffic replay remain future work.

3.3 Abstractification Workflow

§3.1 illustrates abstractification and §3.2 presents how to produce a verifier for leveraging abstractification, but we are yet to talk about how the users write abstractified programs and how the models synthesize concrete implementations.

Abstractification is an inherently human-in-the-loop activity as it involves defining what the specification of a program is in order to allow a greater space of autonomous optimizations. We envision both abstractification of existing software via an `/abstractify` skill similar to `/simplify`[Anthropic, 2026a] skills already existing in agentic coding tools; and a post-hook that goes through model synthesized code to identify potential abstractifications and holes to be part of the code reviewed and approved by the user.

In terms of synthesis, the current implementation uses a simple incremental loop that runs a number of times and concretizes the best accepted implementation; we envision future implementations incorporating AlphaEvolve-style[Novikov et al., 2025] genetic multi-head synthesis. To prevent reward hacking, the autonomously running synthesis agent is not allowed edit programs at all; it can only propose contents for holes which is written by a simple editing tool.

4 Experimentation

We designed a small-scale experiment that asks the model to decide on the reserved capacity for `str::replacen` (Fig. 4). The original constant, 32, was introduced in 2016 as an optimization heuristic; its effect on the performance of the function is distribution-dependent. We adapted `replacen` in our evaluation crate, marked the constant as an expression hole (`String::with_capacity(hole!(32))`) and set up two optimization passes over 9 different input distributions presented in Table 1.

Table 1: Nine input distributions for the `replacen` capacity hole, spanning input size, match density, and output growth.

Distribution	Input	Matches	Output/input	Calls
<code>tiny_notes</code>	48 B	4	1.00×	200k
<code>no_match_scan</code>	65 kB	0	1.00×	3k
<code>dense_grow</code>	30 kB	10,000	2.33×	400
<code>dense_shrink</code>	28 kB	2,000	0.14×	400
<code>single_edit</code>	511 kB	1	1.00×	1k
<code>config_rewrite</code>	2.6 kB	80	1.15×	8k
<code>unicode_prose</code>	18 kB	1,600	0.91×	2k
<code>heavy_calls_small</code>	10 B	2	1.20×	400k
<code>template_expand</code>	1.0 kB	16	2.01×	10k

We then evaluated a fixed candidate loop over four concretizations of the hole, $(0, 1024, \text{text.len()}, \text{text.len()} * 2)$. Each configuration is compiled in release mode and every workload is timed 30 times after a discarded warmup, on an otherwise idle Apple-silicon laptop (rustc nightly 1.96); we report the median of each run set as a percentage difference from the default (32). Table 2 presents the results.

Table 2: Median time difference (%) of each capacity expression relative to the default `with_capacity(32)`; negative is faster. Bold marks the lowest value in each row.

Distribution	0	1024	<code>text.len()</code>	<code>text.len()*2</code>
<code>tiny_notes</code>	+44.2	-41.3	-37.4	-39.8
<code>no_match_scan</code>	-1.0	-0.1	-0.3	-0.6
<code>dense_grow</code>	+0.2	-0.1	+4.8	-1.3
<code>dense_shrink</code>	+0.2	-1.7	-2.2	-1.9
<code>single_edit</code>	+6.0	-0.0	+6.3	+6.2
<code>config_rewrite</code>	+5.8	-12.4	-11.3	-18.5
<code>unicode_prose</code>	-0.1	-1.5	+1.8	-2.7
<code>heavy_calls_small</code>	+51.0	-7.0	+112.3	+0.3
<code>template_expand</code>	+4.3	-50.9	-35.5	-50.8

Finally, we have implemented a model-driven optimization pipeline, using Claude Fable 5 [Anthropic, 2026b] as the proposal model. At the beginning, the model is given the hole, the context of the optimization and input distribution to optimize for, and is asked to produce an expression to substitute for the hole. We implement the edit, run the benchmarks against the input distribution and record the result. This process is repeated six times, with the results of all previous iterations provided in the model’s context each time. The model is invoked once per iteration with no tools and no state beyond that context, so the loop, its budget, and the record of measurements are held by the driver rather than by the model.

The results of the model-driven search loop are presented in Fig. 6, nine workloads with six proposals each; a proposal is accepted when its median improves on the current expression by more than 2% with two-sided Mann–Whitney significance at $p < 0.05$. At least one optimization was accepted for every workload except `no_match_scan`, where the loop kept 32. On `template_expand` the first proposal already finds an exact expansion formula and later proposals settle on the equivalent constant 2048. One proposal was rejected by the correctness verifier rather than by timing: an unclamped growth formula whose capacity saturates at `count = usize::MAX`, caught by the synthesized differential tests (the cross in Fig. 6).

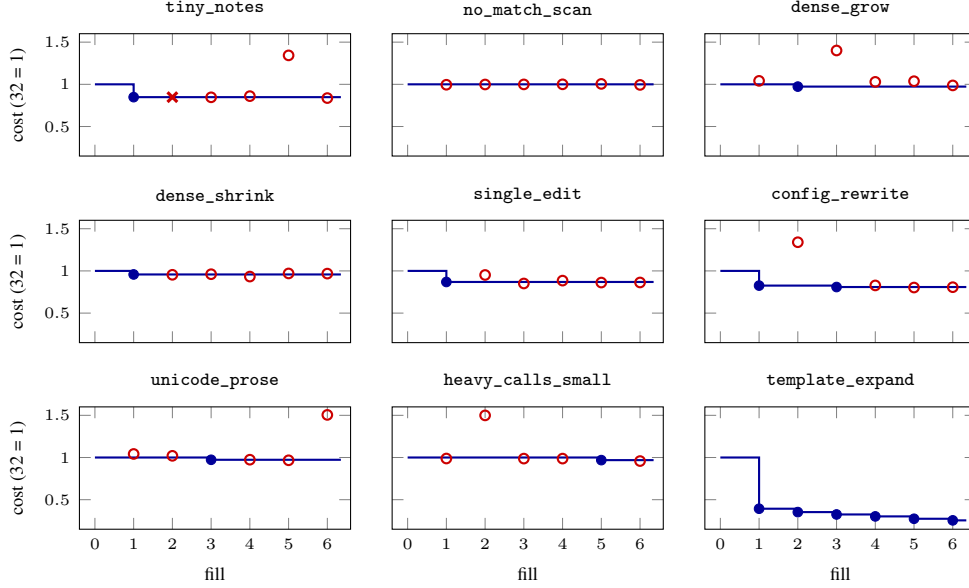


Figure 6: Model-driven search loop measurements against the default capacity (32): filled dots (●) are accepted proposals, open circles (○) are rejected proposals, the cross (×) is a proposal rejected for correctness.

5 Related Work

Abstractification relates to several existing practices already used for autonomous programming.

Test-driven development. Test-driven development (TDD) writes tests before or alongside an implementation and uses them to guide incremental design [Janzen and Saiedian, 2005]. From a synthesis perspective, a test suite is an executable but generally partial specification: passing the tests constrains candidate implementations, but does not establish behavioral equivalence outside the tested cases. Abstractification adopts this separation while making the synthesis boundary explicit and combining tests with other verifiers, such as contracts, reference implementations, and property-based tests.

Program translation. Reference implementations can also serve as executable specifications for translating software between languages. TransCoder, for example, automates source-to-source translation between C++, Java, and Python [Lachaux et al., 2020]. One possible verifier for translated programs is differential testing, which compares implementations on shared generated inputs [Yang et al., 2011]. At a larger scale, Bun’s Zig-to-Rust rewrite illustrates a related agent-assisted but human-supervised migration, retaining tests and human review throughout the process [Sumner, 2026].

Language embeddings. The ability to program at multiple levels of granularity or abstraction have been useful and interesting to programmers for a long time. Many mainstream languages allow for writing inline assembly programs, some allow for mixing statically and dynamically typed programs, many have safety boundaries like Rust’s `unsafe`, Java’s `sun.misc.Unsafe` or Haskell’s `System.IO.Unsafe`. Abstractification is another approach to heterogeneous granularity via providing higher level synthesis primitives.

6 Conclusion and Future Work

The ideas and methods proposed in this paper are still in their infancy, we are merely testing the initial steps of higher level programming languages in the context of agentic development. Using the

sketches as the higher level language, we posit that model-authored autonomous optimizations will allow for previously infeasible level of performance engineering due to the attention bottleneck. In essence, every program can be treated a reference implementation of a faster version of itself, but the autonomous optimization space is too small to be a continuous, closed loop.

Of course, in order to use these autonomous optimizations at scale in production, we will need to work through many challenges. The autonomous formal verification techniques are not yet that reliable[Bursuc et al., 2025], Hyrum’s Law[Winters et al., 2020] means that given enough users, every observable behavior of a system will be depended upon by somebody. Consequently, the space we give to the autonomous optimizer may break existing user experiences.

Treating the existing end-to-end program behavior constant and reorganizing the internal data representations, vectorization, space optimizations, even post-compilation assembly optimization are within the horizon of this line of work. We view models as an augmentation and complement of the existing compilation and optimization techniques.

References

- Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*, pages 1–8. IEEE, 2013. doi: 10.1109/FMCAD.2013.6679385. URL <https://doi.org/10.1109/FMCAD.2013.6679385>.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. arXiv preprint, 2016. URL <https://doi.org/10.48550/arXiv.1606.06565>.
- Anthropic. Claude Code commands. Claude Code documentation, 2026a. URL <https://code.claude.com/docs/en/commands#all-commands>. Entry for /simplify; accessed 2026-08-28.
- Anthropic. Claude Fable 5 and Claude Mythos 5. Anthropic news announcement, June 2026b. URL <https://www.anthropic.com/news/claude-fable-5-mythos-5>. Published 2026-06-09; accessed 2026-08-28.
- Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1–12. ACM, 2000. doi: 10.1145/349299.349303. URL <https://doi.org/10.1145/349299.349303>.
- Mike Barnett, K. Rustan M. Leino, and Wolfram Schulte. The Spec# programming system: An overview. In *Construction and Analysis of Safe, Secure, and Interoperable Smart Devices*, volume 3362 of *Lecture Notes in Computer Science*, pages 49–69. Springer, 2005. doi: 10.1007/978-3-540-30569-9_3. URL https://doi.org/10.1007/978-3-540-30569-9_3.
- Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Ionel Emilian Chiosa, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, Miranda Zhao, Max Tan, and Max Tegmark. A benchmark for vericoding: Formally verified program synthesis. arXiv preprint, 2025. URL <https://doi.org/10.48550/arXiv.2509.22908>.
- Koen Claessen and John Hughes. QuickCheck. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279. ACM, 2000. doi: 10.1145/351240.351266. URL <https://doi.org/10.1145/351240.351266>.
- Ole-Johan Dahl and Kristen Nygaard. SIMULA: An ALGOL-based simulation language. *Communications of the ACM*, 9(9):671–678, 1966. doi: 10.1145/365813.365819. URL <https://doi.org/10.1145/365813.365819>.
- Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. arXiv preprint, 2017. URL <https://doi.org/10.48550/arXiv.1703.07469>.

- Robert Bruce Findler and Matthias Felleisen. Contracts for higher-order functions. In *Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming*, pages 48–59. ACM, 2002. doi: 10.1145/581478.581484. URL <https://doi.org/10.1145/581478.581484>.
- Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM SIGPLAN Notices*, 46(1):317–330, 2011. doi: 10.1145/1925844.1926423. URL <https://doi.org/10.1145/1925844.1926423>.
- Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017. doi: 10.1561/25000000010. URL <https://doi.org/10.1561/25000000010>.
- David Janzen and Hossein Saiedian. Test-driven development: Concepts, taxonomy, and future direction. *Computer*, 38(9):43–50, 2005. doi: 10.1109/MC.2005.314. URL <https://doi.org/10.1109/MC.2005.314>.
- Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, pages 215–224. ACM, 2010. doi: 10.1145/1806799.1806833. URL <https://doi.org/10.1145/1806799.1806833>.
- Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–34, 2017. doi: 10.1145/3158154. URL <https://doi.org/10.1145/3158154>. Article 66.
- Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. URL <https://github.com/karpathy/autoresearch>. Accessed 2026-08-27.
- Viktor Kuncak, Mikaël Mayer, Ruzica Piskac, and Philippe Suter. Complete functional synthesis. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 316–329. ACM, 2010. doi: 10.1145/1806596.1806632. URL <https://doi.org/10.1145/1806596.1806632>.
- Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. arXiv preprint, 2020. URL <https://doi.org/10.48550/arXiv.2006.03511>.
- Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *International Symposium on Code Generation and Optimization*, pages 75–86. IEEE, 2004. doi: 10.1109/CGO.2004.1281665. URL <https://doi.org/10.1109/CGO.2004.1281665>.
- Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023. doi: 10.1145/3586037. URL <https://doi.org/10.1145/3586037>.
- Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884. URL <https://doi.org/10.1145/1127878.1127884>.
- K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20. URL https://doi.org/10.1007/978-3-642-17511-4_20.
- Caroline Lemieux, Rohan Padhye, Koushik Sen, and Dawn Song. PerfFuzz: Automatically generating pathological inputs. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 254–265. ACM, 2018. doi: 10.1145/3213846.3213874. URL <https://doi.org/10.1145/3213846.3213874>.

- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi: 10.1145/1538788.1538814. URL <https://doi.org/10.1145/1538788.1538814>.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL <https://doi.org/10.1126/science.abq1158>.
- Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79. ACM, 2021. doi: 10.1145/3453483.3454030. URL <https://doi.org/10.1145/3453483.3454030>.
- David MacQueen. Modules for Standard ML. In *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*, pages 198–207. ACM, 1984. doi: 10.1145/800055.802036. URL <https://doi.org/10.1145/800055.802036>.
- Henry Massalin. Superoptimizer: A look at the smallest program. *ACM SIGARCH Computer Architecture News*, 15(5):122–126, 1987. doi: 10.1145/36177.36194. URL <https://doi.org/10.1145/36177.36194>.
- Mikaël Mayer, Gustavo Soares, Maxim Grechkin, Vu Le, Mark Marron, Oleksandr Polozov, Rishabh Singh, Benjamin Zorn, and Sumit Gulwani. User interaction models for disambiguation in programming by example. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology*, pages 291–301. ACM, 2015. doi: 10.1145/2807442.2807459. URL <https://doi.org/10.1145/2807442.2807459>.
- Bertrand Meyer. Applying “design by contract”. *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279. URL <https://doi.org/10.1109/2.161279>.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint, 2025. URL <https://doi.org/10.48550/arXiv.2506.13131>.
- Theofilos Petsios, Jason Zhao, Angelos D. Keromytis, and Suman Jana. SlowFuzz: Automated domain-independent detection of algorithmic complexity vulnerabilities. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2155–2168. ACM, 2017. doi: 10.1145/3133956.3134073. URL <https://doi.org/10.1145/3133956.3134073>.
- proptest-rs contributors. proptest 1.11.0 reference documentation. Software documentation, 2026. URL <https://docs.rs/proptest/latest/proptest/>. Accessed 2026-08-27.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2023. doi: 10.1038/s41586-023-06924-6. URL <https://doi.org/10.1038/s41586-023-06924-6>.
- Armando Solar-Lezama. *Program Synthesis by Sketching*. Ph.d. dissertation, University of California, Berkeley, 2008. URL <https://people.csail.mit.edu/asolar/papers/thesis.pdf>. Technical Report UCB/EECS-2008-176.
- Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit A. Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 404–415. ACM, 2006. doi: 10.1145/1168857.1168907. URL <https://doi.org/10.1145/1168857.1168907>.

- Sonar. State of code developer survey report 2026. Industry survey report, 2026. URL <https://www.sonarsource.com/state-of-code-developer-survey-report.pdf>. Accessed 2026-08-27.
- Jarred Sumner. Rewriting Bun in Rust. Bun Blog, July 2026. URL <https://bun.com/blog/bun-in-rust>. Accessed 2026-08-27.
- Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 60–76. ACM, 1989. doi: 10.1145/75277.75283. URL <https://doi.org/10.1145/75277.75283>.
- Titus Winters, Tom Manshreck, and Hyrum Wright. Software engineering at google, chapter 1: What is software engineering? Online book chapter, 2020. URL <https://abseil.io/resources/swe-book/html/ch01.html>. Section “Hyrum’s Law”; accessed 2026-08-27.
- Chenyang Yang, Yike Shi, Qianou Ma, Michael Xieyang Liu, Christian Kästner, and Tongshuang Wu. What prompts don’t say: Understanding and managing underspecification in LLM prompts. arXiv preprint, 2025. URL <https://doi.org/10.48550/arXiv.2505.13360>.
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 283–294. ACM, 2011. doi: 10.1145/1993498.1993532. URL <https://doi.org/10.1145/1993498.1993532>.
- Zack Zeng and Rust Project Contributors. Implement `std::str::replacen`. Rust source code, merge commit `ebda77072a56a43d33d5723196a5ae37544a1ab9`, 2016. URL <https://github.com/rust-lang/rust/blob/ebda77072a56a43d33d5723196a5ae37544a1ab9/src%2Flibcollections%2Fstr.rs>. Introduced by rust-lang/rust pull request 36347; accessed 2026-08-27.